Resuelve los siguientes ejercicios y entrega el código fuente java en un fichero .pdf y en ficheros de texto, en el .pdf capturas de pantalla (si se piden) y las respuestas a las preguntas que se plantean, si las hay.

EJERCICIOS PADAWAN (OBLIGATORIOS)

EJERCICIO 1. Crea/Usa una máquina virtual que tenga un SO Windows (Windows 7, 8, 10) de 64 bits. Instala y configura el SDK de Java jdk12.* (el asterisco significa cualquier versión) y me pasas capturas de pantalla de:

- Ejecución de la instalación (o comando donde se vea la versión tras instalar).
- Definición de las variables de entorno.
- Crea, compila y ejecuta un programa donde aparezca en consola el mensaje: "Hola Jose, soy <tu nombre y apellidos> y estoy en Windows"

EJERCICIO 2. Crea/Usa una máquina virtual que tenga un SO Windows GNU/Linux de 64 bits (Ubuntu, Debian...). Instala y configura el SDK de Java, preferiblemente jdk12.* (el asterisco significa cualquier versión) o bien Openjdk si el otro te da problemas. Me pasas capturas de pantalla de:

- Operaciones previas a la instalación: actualizaciones, etc.
- Ejecución de la instalación (donde se vea la versión).
- Definición de las variables de entorno.
- Crea, compila y ejecuta un programa donde aparezca en consola el mensaje: "Hola Jose, soy <tu nombre y apellidos> y estoy en Linux!!"

EJERCICIO 3. USO DE LA CLASE PRECONSTRUIDA String.
Define tres variables de tipo String, una llamada USERNAME, otra PASSWORD y otra VALIDACLAVE (sigue las convenciones de nomenclatura en el programa Java). Pregunta las dos primeras al usuario y a la tercera la inicializas a la cadena vacía. No podrás usar sentencias de control (if-else, while, do-while, etc. Porque no sabemos que existen) pero sí puedes usar los operadores que quieras (INCLUIDO EL TERNARIO) para:

- Si el usuario tecleado por el usuario coincide con "admin", añade a validaclave "El usuario "admin" no se puede utilizar" (las dobles comillas deben aparecer en el mensaje) y un salto de línea. De lo contrario añades la cadena vacía (nada).
- Ahora comprueba que el usuario no comience por número, si lo hace añade a validaclave "El nombre de usuario debe comenzar por letra" y un salto de línea.
- Luego comprueba si el password tiene al menos 8 caracteres de longitud, sino, añades a la cadena validaclave "Password debe tener un mínimo de 8 caracteres" y un salto de línea.
- Ahora comprueba que tenga al menos una minúscula. Si no es así, añade a validaclave la cadena "El password debe tener una letra minúscula" y un salto de línea.
- Ahora comprueba que tenga al menos una mayúscula. Si no es así añade a la cadena validaclave la cadena "El password debe tener una letra mayúscula" y un salto de línea.

- Como última comprobación, chequea que tenga al menos una letra "." o "_" o "@". Si no es así añade a la cadena validaclave la cadena "El password debe tener al menos un símbolo "._@"" (las comillas dobles deben aparecer en el mensaje por pantalla) y un salto de línea.

Para acabar, imprime la cadena validaclave. Dos ejecuciones de ejemplo:

```
Nombre de usuario: admin
Password: Limon@d@
  El usuario "admin" no se puede utilizar
```

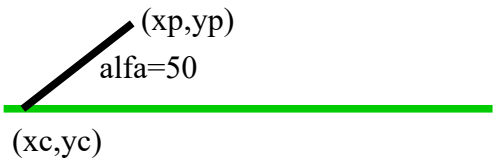
```
Nombre de usuario: 2Jose
Password: Limon
  El nombre de usuario debe comenzar por letra
  El password debe tener al menos una letra "._@"
```

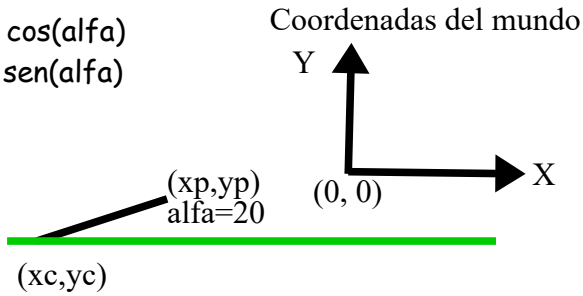
EJERCICIO 4. LA CLASE PRECONSTRUIDA Math Y FORMATEO DE TEXTO CON printf.
Queremos hacer un juego 2D en el que un cañón cuya base está en las coordenadas (xc, yc) y tiene una longitud constante llamada LONGITUD_CANION en píxels, dispara proyectiles que salen desde unas coordenadas de la pantalla (xp, yp). Al cañón podemos cambiarle:

- El ángulo de inclinación (lo llamamos alfa).
- La velocidad inicial a la que dispara los proyectiles (Vi) en m/s.

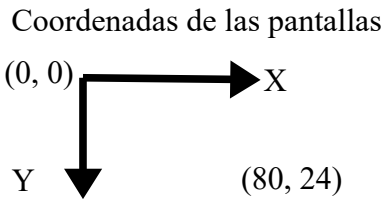
Como la posición del cañón (xc, yc) no cambia, la posición (xp,yp) desde donde sale el proyectil cambiará solamente cuando cambie alfa. La posición del extremo del cañón en el mundo (xp, yp) la puedes calcular como:

xp= xc + LONGITUD_CANION * cos(alfa)
yp= yp + LONGITUD_CANION * sen(alfa)





El proyectil debe dibujarse en una pantalla a medida que se desplaza por el aire. La pantalla queda descrita por un sistema de coordenadas 2D, donde la esquina superior izquierda es la coordenada (0,0) y la inferior derecha es la (80, 24). Es decir, al contrario de lo que ocurre con los sistemas de coordenadas que se usan en matemáticas o física, en la pantalla el eje Y (vertical) crece hacia abajo.



Otro inconveniente que tendremos es que en el mundo real el espacio es contínuo (el proyectil puede estar en la posición (30, 40), (30.5, 40.83), ..., etc. Pero en la pantalla, las coordenadas son enteras (o píxels-puntos iluminables- si estamos en modo gráfico o caracteres en modo texto). Por tanto habrá que transformar (escalar) la posición del mundo (valores en coma flotante) a la de pantalla (valores enteros). Imaginemos que cada posición son 5 metros.

El movimiento parabólico (la trayectoria que sigue un proyectil) se descompone en dos movimientos para comprenderlo mejor:

- **El movimiento horizontal:** es rectilíneo y uniforme (no acelera), el objeto siempre se mueve a la misma velocidad (en realidad no por el rozamiento, pero simplificamos). La velocidad horizontal (no cambia) se obtiene al multiplicar Vi por el coseno del ángulo alfa.
- **El movimiento vertical:** es rectilíneo uniformemente acelerado, el proyectil va cambiando su velocidad de forma constante. Al principio sale del cañón a cierta velocidad inicial (Vi). Parte de ella se aplica en el eje vertical y se calcula multiplicando Vi por el seno del ángulo alfa. Debido a la gravedad, a medida que gana altura, va perdiendo velocidad hasta alcanzar su altura máxima (hmax) y cuando la pierde toda, se para (su velocidad vertical es cero), en ese momento, comienza a acelerarse de nuevo a medida que cae, debido a la gravedad.

Conociendo este comportamiento y las leyes del movimiento podemos calcular:

- **El tiempo de subida (ts)**, los segundos que tarda en alcanzar la altura máxima. Si sabemos ts, calculamos el tiempo de vuelo (tv) que es el doble.
- **La distancia horizontal máxima** que va a recorrer (Xmax).
- **Posición horizontal en cualquier momento de tiempo (t)**, comprendido entre 0 (cuando se dispara) y tv (cuando cae al suelo), podemos calcular la posición horizontal (su coordenada x) en el mundo.

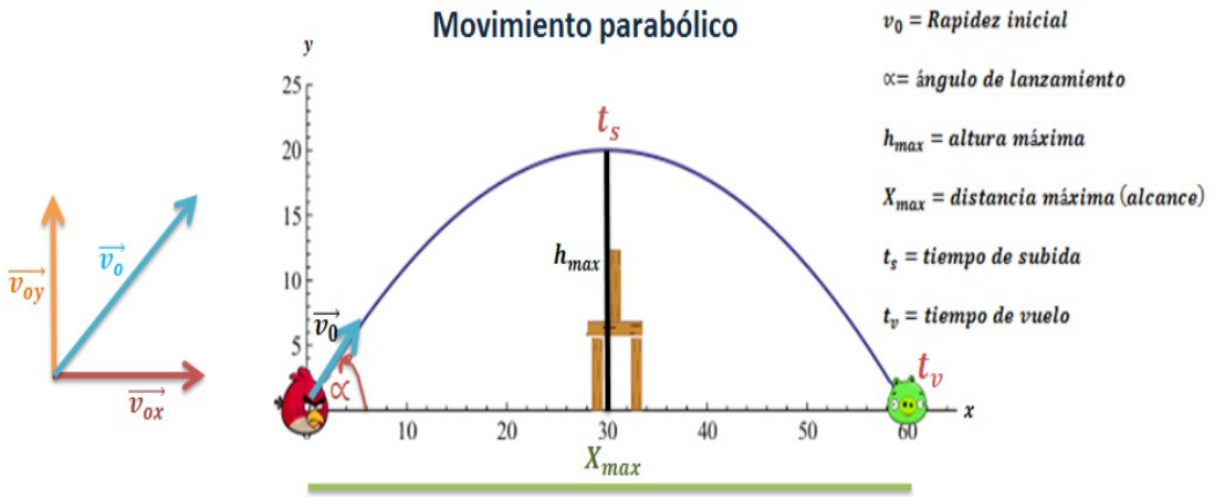
  x(t) = xp + Vix * t = xp + Vi cos(alfa) * t

- **Y también podemos saber su posición vertical en cualquier instante t**, pues la altura respecto del cañón viene dada por:

  y(t) = yp + Viy * t – 0.5 * g * t * t = yp + Vi * sen(alfa) * t – 0.5 * g * t * t

  donde g es la aceleración de la gravedad (9.8 m/s^2).

Estas figuras contienen alguna de las fórmulas que debes emplear (cambian algunas letras).



En el eje x:

MRU

$$v_{0x} = v_0 \cos \propto$$

$$x = v_{0x}\, t$$

$$x_{max} = \frac{v_o^2 \sin 2 \propto}{g}$$

$$t_v = 2\, t_s$$

$$\overrightarrow{v_0} = \overrightarrow{v_{0x}} + \overrightarrow{v_{0y}}$$

$$Y = -\frac{g}{2v_0^2 \cos^2 \propto} x^2 + x \tan \propto$$

En el eje y:

MRUV

$$v_{0y} = v_0 \sin \propto$$

$$y = v_{0y}\, t - \frac{1}{2}\, g\, t^2$$

$$h_{max} = \frac{v_o^2 \sin^2 \propto}{2g}$$

$$t_s = \frac{v_0 \sin \propto}{g}$$

Haz un programa que defina las constantes posición del cañón en el mundo en (0,0) y su longitud en el mundo a 5 metros. También define la posición inicial en la pantalla a (0,24).
Luego debe pedir el ángulo alfa en grados y la velocidad inicial del proyectil en metros/segundo al usuario.
Con estos datos debe calcular y mostrar la posición inicial del proyectil, el tiempo de subida y de vuelo en segundos, la distancia horizontal máxima que recorrerá y la vertical. Luego debe crear una tabla donde todos los valores reales tengan una anchura de 10 y 2 decimales, y los enteros tengan una anchura de 5 caracteres:

```
t= 0                coordenadas en el mundo          coordenadas en pantalla
t=25% del tv        coordenadas en el mundo          coordenadas en pantalla
t=50% del tv        coordenadas en el mundo          coordenadas en pantalla
t=75% del tv        coordenadas en el mundo          coordenadas en pantalla
t=100% del tv       coordenadas en el mundo          coordenadas en pantalla
```

Ejemplo de ejecución:

Los dígitos en verde y todos los de la tabla, necesitan un ancho fijo

```
F:\P01>java E4.java
Teclee angulo (grados): 30
Teclee velocidad (m/s): 20
  Pos. inicial del proyectil: ( 4,33, 2,50)
  Tiempo de subida:        1,02 segundos y de vuelo 2,04 segundos
  Max. altura:             5,10 metros
  Max. distancia:         35,35 metros
+-------------+---------------------------+----------------------+
| Tiempo      |   Posicion en el mundo    | Posicion en Pantalla |
+-------------+---------------------------+----------------------+
|       0,00  | (      4,33 ,       2,50) |    (  0, 24)         |
+-------------+---------------------------+----------------------+
|       0,51  | (     13,17 ,       6,33) |    (  2, 23)         |
+-------------+---------------------------+----------------------+
|       1,02  | (     22,00 ,       7,60) |    (  4, 23)         |
+-------------+---------------------------+----------------------+
|       1,53  | (     30,84 ,       6,33) |    (  6, 23)         |
+-------------+---------------------------+----------------------+
|       2,04  | (     39,68 ,       2,50) |    (  7, 24)         |
+-------------+---------------------------+----------------------+
```

EJERCICIO 5. MÁS FORMATOS DE SALIDA CON PRINTF().

Un profesor está calificando unas prácticas en una escala de 0 a 10. Cada práctica tiene dos partes: un cuestionario que tiene un peso (de 0 a 1) y una actividad que tiene otro peso (0-1).

Haz un programa que pregunte por los pesos del cuestionario y de la actividad y calcule la nota de la práctica. Si la suma de los dos pesos no da 1, debe mostrar un mensaje de advertencia, si coinciden, el mensaje de aviso estará vacío (no uses if-else, usa el operador ternario para dar valor al mensaje).

Pero hay otro problema, la nota debe subirla a una plataforma. Esta plataforma califica las prácticas en una escala de 10 a 50. Haz el cambio de escala de [0,10] a [10,50].

> **Nota**: *para hacer un cambio de escala de un valor x dentro de un intervalo [a,b] a otro intervalo [c,d] debes usar esta fórmula, que básicamente lo que hace es calcular en qué % de la distancia (b-a) queda el valor y calcular el valor que le corresponde en la distancia (d-c):*

$$nx = c + \frac{x-a}{b-a}(d-c)$$

Tras calcular y mostrar la nota en [0,10] y en otra línea calcula y muestra la nota en [10, 50]. Todas las notas deben tener dos decimales y quedar alineadas (usar printf).

```
C:\>java E5.java
Teclee peso del cuestionario (0.0-1.0): 0.4
Teclee peso de la actividad (0.0-1.0): 0.6

Nota de cuestionario en [0-10]: 5
Nota de actividad en [0-10]: 5
Nota en [0,  10] es  5.00
Nota en [10, 50] es 30.00
```

```
C:\>java E5.java
Teclee peso del cuestionario (0.0-1.0): 0.4
Teclee peso de la actividad (0.0-1.0): 0.4
  ERROR EN PESOS!!
Nota de cuestionario en [0-10]: 5
Nota de actividad en [0-10]: 5
Nota en [0,  10] es  4.00
Nota en [10, 50] es 26.00
```

EJERCICIOS JEDI (EL QUE TE TOQUE)

Como siempre, aunque solo tenéis que entregar el que os toca, deberíais intentar hacerlos todos para practicar.

EJERCICIO 6. USO DE LA CLASE PRECONSTRUIDA System.

a) Para programar algunas acciones, influye el entorno donde se ejecuta el programa. Por este motivo, sería buena idea que nuestros programas fuesen capaces de detectar el SO donde se están ejecutando. Haz un programa que pruebe este código tanto en Windows como en GNU/Linux e imprime "Me ejecuto en <Windows/Linux/Otro>" según el caso:

```
String so = System.getProperty("os.name");
```

b) Una de las cosas que dependen del entorno, es por ejemplo algo tan sencillo como borrar la pantalla de un terminal. Aquí tienes varias opciones para hacerlo. Prueba todas tanto en Windows como en GNU/Linux a ver si funcionan y haz una marca (pág. siguiente):

Opción a) Ejecutar un comando del sistema.	`// En Windows el programa terminal es "cmd" en GNU/Linux ¿?`
	`// En Windows el comando cls borra la terminal en GNU/Linux es clear`
	`try {`
	`  new ProcessBuilder("programa_terminal", "/c", "comando").inheritIO().start().waitFor();`
	`} catch (Exception e) {  /* No hacer nada*/ }`

Opción b) Envia secuencia escape ANSI al terminal	`System.out.print( "\033[H\033[2J" );` `System.out.flush();`

Opción A: ☐ **Funciona en Windows.** ☐ **Funciona en GNU/Linux.**
Opción B: ☐ **Funciona en Windows.** ☐ **Funciona en GNU/Linux.**

c) Utiliza la documentación de Java API y busca información de otro método de la clase System que permita medir cuantos milisegundos han pasado desde un determinado momento. Indica qué método es, cómo se llama y lo que devuelve.

d) Aprovechando lo que has aprendido en los apartados anteriores, haz un programa que detecte el SO (Windows o GNU/Linux), borre la pantalla de alguna forma que funcione y muestre el mensaje "Soy <tu nombre y apellidos> ¿Cuánto es <n1> * <n2>?". Donde n1 y n2 son dos números enteros aleatorios entre 0 y 20. Debe calcular cuanto tiempo tarda el usuario en responder y mostrar si ha acertado o fallado y el tiempo empleado en contestar en milisegundos.

EJERCICIO 7. CLASES PARA FECHAS.

Repasa las clases para manejar fechas y úsalas para crear este programa. Un ciudadano está haciendo un trámite en una determinada fecha y le dan una serie de días naturales para presentar una documentación. Haz un programa que pregunte por la fecha en la que hace el trámite y el número de días naturales que tiene de plazo y te devuelva la fecha tope en la que debes entregar la documentación (si es sábado o domingo debe retroceder de día). Ejemplo:

```
Día del mes (1-31): 14
Mes del año (1-12): 12
Año: 2020
Plazo (dias naturales): 20

Debe entregarlo hasta el viernes 1 de enero de 2021
```

EJERCICIO 8. EL OPERADOR TERNARIO.

Escriba un programa en Java que pregunte al usuario por la velocidad a la que conduce (kms/h) y calcule el importe de la multa por exceso de velocidad en una autopista, pero **sin usar sentencias de control de tipo if-else**. Las reglas son las siguientes:

- De 0 a 60: 300,00€ por ir debajo del límite.
- Más de 60 hasta 120: no hay multa (importe 0,00).
- Más de 120 hasta 140 incluido: 300,00€.
- Más de 140 hasta 180: 600,00€.
- Más de 180: 1000,00€.

Nota: *si intentas anidar sentencias del tipo: (c) ? v : f; te dará error de compilación salvo que encierres entre paréntesis la sentencia anidada:*

```
(1 < 2)? -1:(1==2) ? 0:1;  → error
(1 < 2)? -1:((1==2) ? 0:1);  → Ok!
```

Ejemplo de ejecución:

```
Velocidad (Kms/h): 130
Multa: 300,00
```

EJERCICIO 9. LA CLASE BIGINTEGER.

Piensa/busca en los apuntes los bits que tiene un tipo long en Java y teniendo en cuenta que este tipo usa el formato complemento a 2 para almacenar los números enteros, define uno y almacena el mayor número impar que puede contener (recuerda que puedes definir valores literales no solo en decimal). Lo imprimes y repasando y usando la clase BigInteger, piensa como encontrar los dos números probablemente primos, que sean mayores que el mayor impar positivo que has definido antes.

```
P01>java E9.java
El mayor long impar es:              9223372036854775807
El primer primo probable mayor es    9223372036854775837
El siguiente primo probable mayor es 9223372036854775907
```

EJERCICIO 10. LA CLASE STRINGBUILDER.

Un programa quiere crear una sentencia SELECT de SQL que pueda filtrar filas por las columnas <u>apellido</u> y <u>edad</u> para seleccionar todos los datos de la tabla <u>EMP</u>. Inicialmente la consulta contiene el texto "select * from emp".

Haz un programa que lea como comienza el apellido. Si el usuario no indica nada para el apellido la sentencia queda así:

"select * from emp"

Si indica algo queda:

"select * from emp where apellido LIKE 'algo*' "

Y pregunta por una edad. Si no indica nada, la sentencia no cambia. Para no indicar nada, debes leer un String. Si indica una edad x, podrían pasar dos cosas:

- Si ya tiene el where, debe añadir " and edad = x"
- Si no tiene el where, piensa que deberías añadir.

Por último imprime la sentencia. Como siempre, para nosotros aún no existen las sentencias de control. Ejemplos:

```
El apellido comienza por: San
Edad:

Select * from emp where apellido like 'San*'
```

```
El apellido comienza por:
Edad:

Select * from emp
```

```
El apellido comienza por:
Edad: 30

Select * from emp where edad = 30
```

```
El apellido comienza por: lo
Edad: 40

Select * from emp where apellido like 'lo*' and edad = 40
```